

Elite Sports Club - Infraestructura Serverless en AWS

Descripción del Proyecto

Este proyecto implementa una arquitectura serverless en AWS para gestionar las acciones de los socios de un club de barrio. Las acciones incluyen:

- Registro de usuarios mediante autenticación con Cognito.
- Reservas de instalaciones (una por tipo: fútbol, tenis, yoga, etc.).
- Control de disponibilidad mediante funciones Lambda conectadas a una tabla DynamoDB.
- Sitio web estático para la interfaz del usuario hospedado en S3.

Estructura del Proyecto

...

club-sports/

— main.tf	# Orquesta los recursos y módulos principales
— datasources.tf	# Define datasources como archive_file
— output.tf	# Define outputs generales (ARNs, endpoints, etc.)
— providers.tf	# Configuración del provider AWS
— terraform.tfvars	# Valores concretos de las variables usadas
— variables.tf	# Variables globales
— version.tf	# Define la versión mínima requerida de Terraform
— lambda_functions/	# Contiene funciones Lambda en Python
— crearReserva.py	# Inserta reservas en DynamoDB a partir del email del usuario autenticado
— getReservas.py	# Consulta reservas existentes por instalacion, fecha u otros filtros
— redirect.py	# Redirige a una URL (ej. para login o validación)
— redirectBucket.py	# Similar a redirect, pero para flujos de navegación específicos del sitio
— registrar_usuario.py	# Se ejecuta post-confirmación en Cognito para registrar el usuario en DynamoDB
— output_lambda_functions/	# Archivos .zip generados automáticamente desde funciones Lambda
	# Necesarios para su despliegue en AWS
	# Se generan con archive_file y contienen el código Python comprimido
	# Esto asegura que el código esté empaquetado correctamente y compatible con Lambda
— modules/	# Módulos reutilizables de Terraform
— apigateway/	# Define recursos de API Gateway
— cognito/	# Crea el user pool, client, dominio y autorizador
— cognito/dynamodb/	# Tabla DynamoDB para almacenar usuarios y reservas
— dynamodb_endpoint/	# Endpoint privado para acceso a DynamoDB desde la VPC
— lambdas/	# Despliegue de funciones Lambda y configuraciones
— s3/	# Sitio web estático hospedado en S3 con acceso público
— vpc/	# Crea la red VPC con subnets y tablas de ruteo
— site/	# Frontend del proyecto (sitio web estático)
— index.html	# Página principal del sitio
— login.html	# Página de login
— profile.html	# Perfil de usuario
— dashboard.html	# Panel de administración
— bookings.html	# Visualización de reservas
— admin.html	# Gestión de instalaciones
— 404.html	# Página de error personalizada
— css/	# Archivos de estilos CSS
— styles.css	# Estilos generales y específicos para el sitio
— js/	# Scripts del sitio (ej: login, redirección, validaciones)

			admin.js	# Gestión de clases e instalaciones
			auth.js	# Lógica de autenticación y autorización con Cognito
			bookings.js	# Visualización y control de reservas desde el frontend
			dashboard.js	# Funcionalidad general del panel de administración
			profile.js	# Muestra y gestiona la información del perfil del usuario
			script.js	# Script general para la navegación y eventos de página
			images/	# Recursos visuales utilizados en el sitio (ilustraciones, íconos, logos, etc.)
			...	

Descripción de los Módulos

Módulo apigateway

Gestiona los recursos y métodos de la API REST de AWS API Gateway.

- Crea rutas como `/redirectBucket`, `/check-availability`, `/create-reservation`.
- Integra con funciones Lambda mediante `AWS\_PROXY`.
- Configura métodos GET, POST y OPTIONS para CORS.
- Usa autenticación con Cognito para proteger ciertas rutas.
- Crea el deployment y stage.

Módulo cognito

Crea los recursos relacionados a Cognito para autenticación:

- `aws\_cognito\_user\_pool`: el pool de usuarios.
- `aws\_cognito\_user\_pool\_client`: aplicación cliente.
- `aws\_cognito\_user\_pool\_domain`: dominio personalizado.
- `aws\_api\_gateway\_authorizer`: authorizer para API Gateway.

Módulo cognito/dynamodb

Define una tabla DynamoDB:

- Con claves primarias PK y SK.
- Índices globales secundarios para múltiples patrones de consulta: por fecha, usuario, instalacion y estado.

Módulo dynamodb_endpoint

Crea un VPC Endpoint tipo Gateway:

- Permite a las Lambdas acceder a DynamoDB desde dentro de la VPC sin salir a internet.

Módulo lambdas

Despliega las siguientes funciones Lambda:

- `registrar\_usuario`: registra el usuario post-confirmación en Cognito.
- `getReservas`: consulta disponibilidad de instalaciones.
- `crearReserva`: inserta una reserva nueva.
- `redirectBucket`: redirecciona a URL de interés.

Incluye configuración VPC (subnet y security group) y variables de entorno.

Módulo S3

Hospeda un sitio web estático:

- Define el bucket S3 y configuración de hosting (index.html y 404.html).
- Permite acceso público para los objetos del bucket.

Módulo VPC

Crea la infraestructura de red:

- VPC con CIDR definido.
- Subnets distribuidas por zona de disponibilidad.
- Tabla de ruteo asociada.

Variables globales y outputs

- Las variables definidas en ``variables.tf`` y sus valores en ``terraform.tfvars`` permiten parametrizar toda la infraestructura.
- Los outputs definidos permiten reutilizar ARN, IDs y URLs entre módulos.

Consideraciones técnicas

- Se utiliza el proveedor AWS versión ``~> 5.0``.
- Las funciones Lambda deben estar previamente empaquetadas en ``.zip`` para ser desplegadas correctamente con Terraform (usando ``archive_file``).
- La arquitectura es totalmente serverless, sin instancias EC2.
- El acceso a DynamoDB es seguro desde la VPC gracias al VPC Endpoint.
- Las rutas de API Gateway están protegidas por Cognito excepto ``redirectBucket``.

Flujo de uso típico

1. El usuario accede al sitio web estático (S3).
2. Se registra o loguea mediante el flujo de autenticación Cognito.
3. Al confirmar el email, se ejecuta la Lambda ``registrar_usuario`` que lo registra en DynamoDB.
4. Luego puede consultar disponibilidad (``getReservas``) y reservar una instalación (``crearReserva``).

Interfaz web (Sitio estático)

El frontend del sistema está desarrollado como un sitio web estático y hospedado en S3. Incluye:

- Páginas públicas y privadas como ``index.html``, ``login.html``, ``dashboard.html``, etc.
- Un sistema de navegación y vistas protegidas que dependen del estado de autenticación del usuario.
- Estilos personalizados definidos en ``styles.css`` y organizados dentro de la carpeta ``css``.
- Interactividad provista por scripts en JavaScript (``js/``) que manejan validaciones, redirecciones y llamadas a la API Gateway.
- Recursos visuales organizados en la carpeta ``images`` para mejorar la experiencia de usuario (íconos, ilustraciones y elementos gráficos).

La integración con Cognito permite restringir acceso a ciertas páginas. Además, las acciones en el sitio (como realizar una reserva o consultar disponibilidad) se traducen en llamadas a funciones Lambda expuestas mediante API Gateway.

Guía de Despliegue con Terraform

Para desplegar toda la infraestructura desde cero, seguí estos pasos dentro de la carpeta raíz ``club-sports/``:

1. Inicializar el proyecto:

Clonar el repositorio:

```
git clone ruta_github
```

Entrar a la carpeta que descarga el repositorio:

```
cd club-sports
```

Actualizar las credenciales de AWS.

```
``bash
terraform init
````
```

2. Verificar el plan de ejecución:

```
```bash
terraform plan
```
```

3. Aplicar el plan para crear los recursos:

```
```bash
terraform apply
```
```

4. Confirmar con `yes` cuando Terraform lo solicite.

Esto levantará automáticamente todos los recursos definidos en los módulos y los integrará correctamente en una arquitectura serverless funcional sobre AWS.

En index.html:

- línea 173: cambiar la api base url (tomarla del lab)
- línea 200: cambiar el client ID (tomado del client dentro del user pool de cognito, tomar el client que tiene el “view login” activado —> buscarlo en el lab)
- línea 201: camnbiar el redirect Uri (este está impreso en la consola)

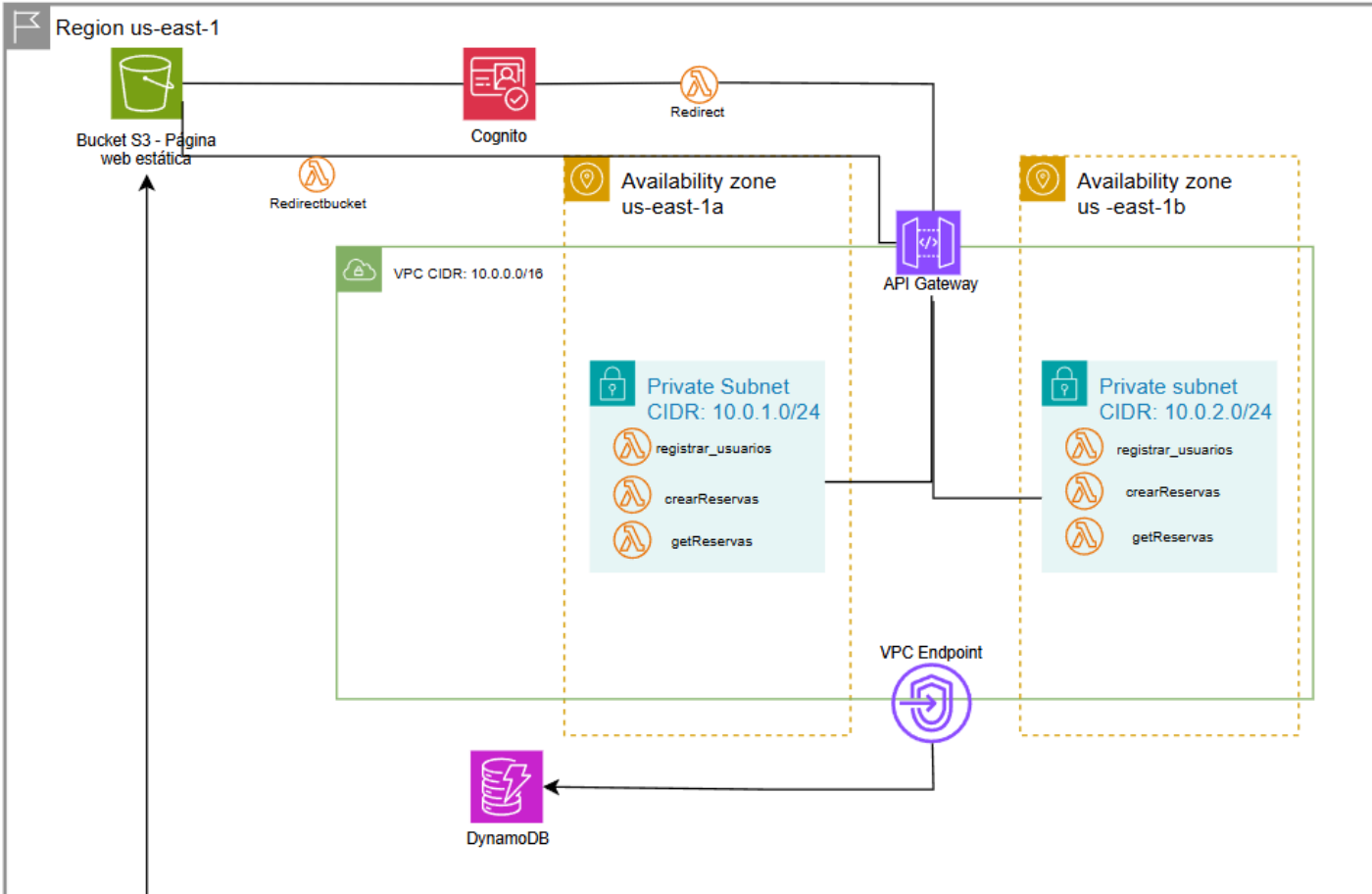
En bookings.js

- línea 1: cambiar la api base url
- línea 7: cambiar el client ID (tomado del client dentro del user pool de cognito, tomar el client que tiene el “view login” activado —> buscarlo en el lab)
- línea 9: camnbiar el redirect Uri (este está impreso en la consola)

En auth.js:

- línea 7: cambiar el client ID (tomado del client dentro del user pool de cognito, tomar el client que tiene el “view login” activado —> buscarlo en el lab) —> dice user id, pero es el ID del client
- línea 9: camnbiar el redirect Uri (este está impreso en la consola)

Todo esto se hace antes de subir los archivos y carpetas manualmente al bucket de S3.



Usuario